# Project Report

## Wildlife Information System (WIS)

Document Number: WIS-SRS-001
Version: 3.0
Date: June 2026

**Prepared by:**
Muhammad Ehtisham
Fahad Khan
Muhammad Abbas

# 3. ANALYSIS

This chapter models the Wildlife Information System (WIS) requirements and the information exchanged between users, external applications, system processes, and persistent data stores. The models are based on the actors, use cases, functional requirements, and data entities described in SRS Version 2.0.

## 3.1 Domain Model

The domain model identifies the principal concepts managed by WIS. Animal is the central entity. Each animal belongs to one species and may have multiple movement records, DNA samples, and health records. A user may be recorded as the person who added an animal. Location is associated with an animal profile. The l ocation attributes are kept at a conceptual level because the SRS does not define a complete Location schema.

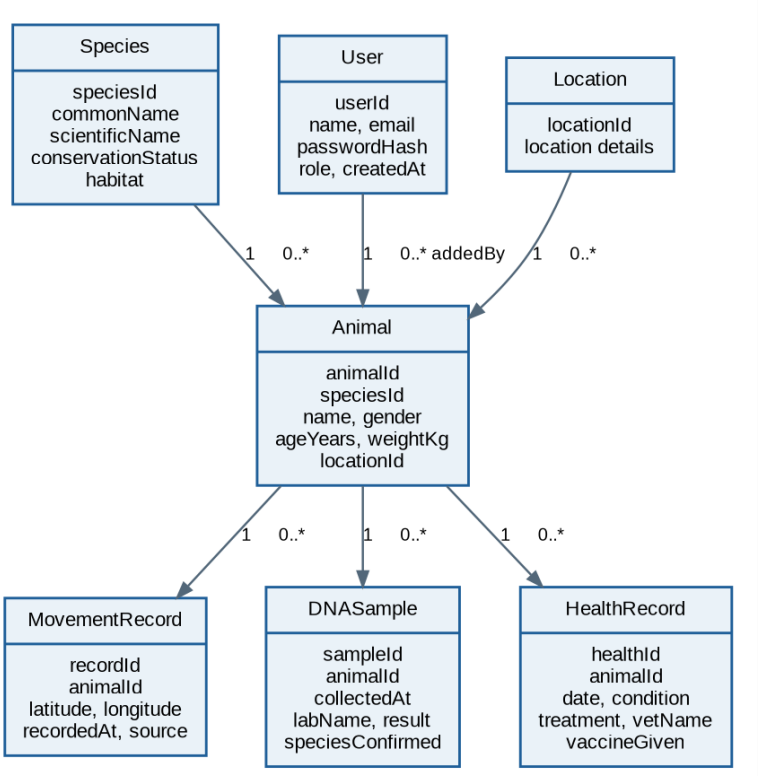



*Figure 3.1: WIS domain model and principal relationships.*

### Relationship interpretation

- Species to Animal: one species may be associated with many animal records; each animal references one species.• Animal to MovementRecord, DNASample, and HealthRecord: one animal may have zero or many records of each type; each record references one animal.
- User to Animal: the addedBy reference identifies the user responsible for entering the animal record.
- Location to Animal: an animal profile references a location; detailed location attributes are not specified in the SRS.

## 3.2 Data Flow Diagrams (DFD)

DFD Level 0 treats WIS as a single process and shows its external interactions. Level 1 decomposes the system into authentication, wildlife data entry, search/API processing, and user administration. The stores correspond to the principal tables named in the SRS.

## DFD Level 0 — Context Diagram

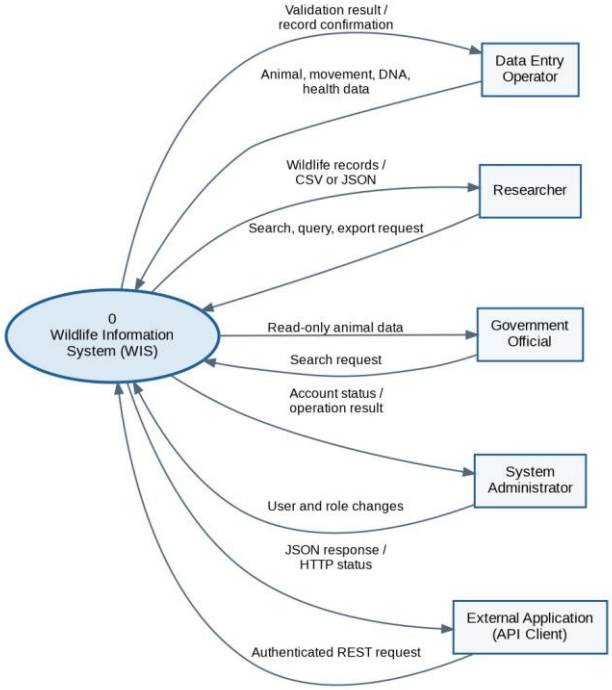



*Figure 3.2: Context-level data flows between WIS and external entities.*

## External entities and exchanged information

Data Entry Operators submit animal, movement, DNA, and health information. Researchers submit search, query, and export requests and receive records in CSV or JSON. Government Officials use read-only search. System Administrators manage accounts and roles. External Applications communicate through authenticated REST API requests and receive JSON responses with  HTTP status codes.

## DFD Level 1 — Main Processes

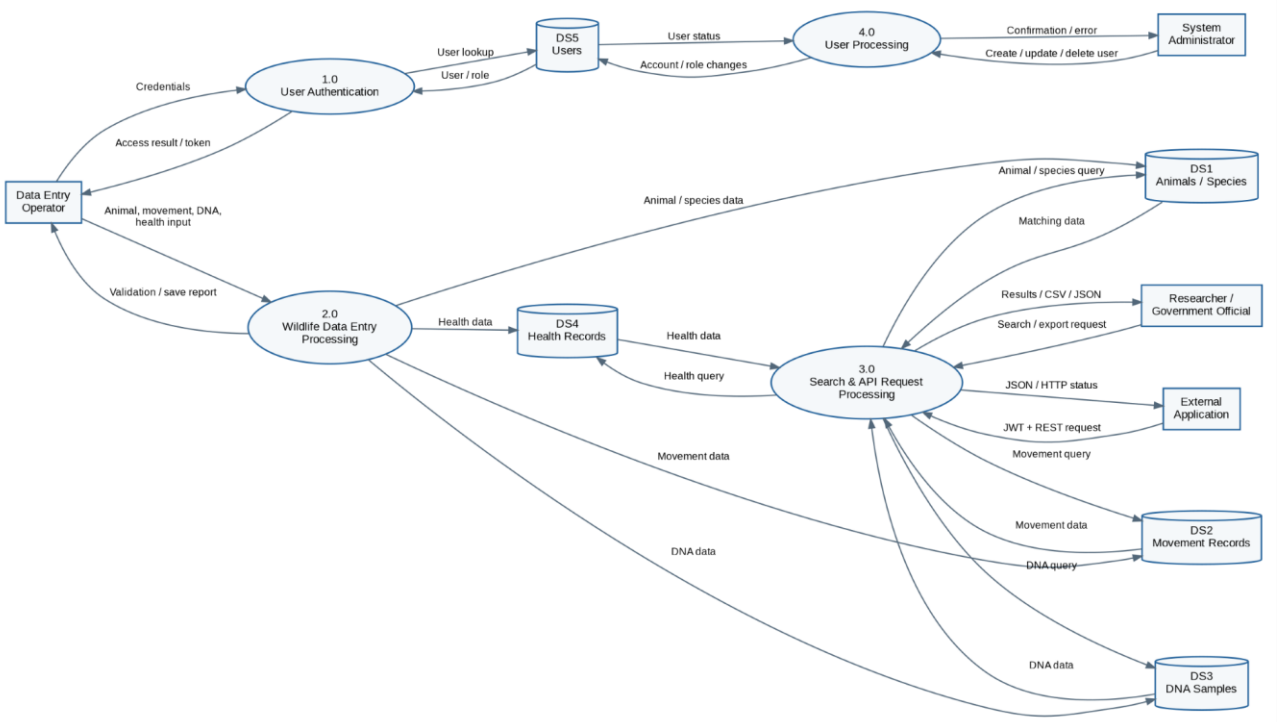



*Figure 3.3: Level 1 DFD showing the main WIS processes and data stores.*

## Process and data-store descriptions

**1.0 User Authentication:** validates credentials, checks the user record and supports token-based access.

**2.0 Wildlife Data Entry Processing:** validates and stores animal, movement, DNA, and health information.

**3.0 Search & API Request Processing:** retrieves matching wildlife information and returns web/API results or export data.

**4.0 User Processing:** allows an administrator to create, update, or delete user account information.

**DS1–DS5:** Animals/Species, Movement Records, DNA Samples, Health Records, and Users respectively.

## 3.3 System Sequence Diagram

The system sequence diagram describes the external view of a key interaction: a researcher requests animal records through the REST API. The system is treated as a single black box; internal MVC components are detailed later in Chapter 4.

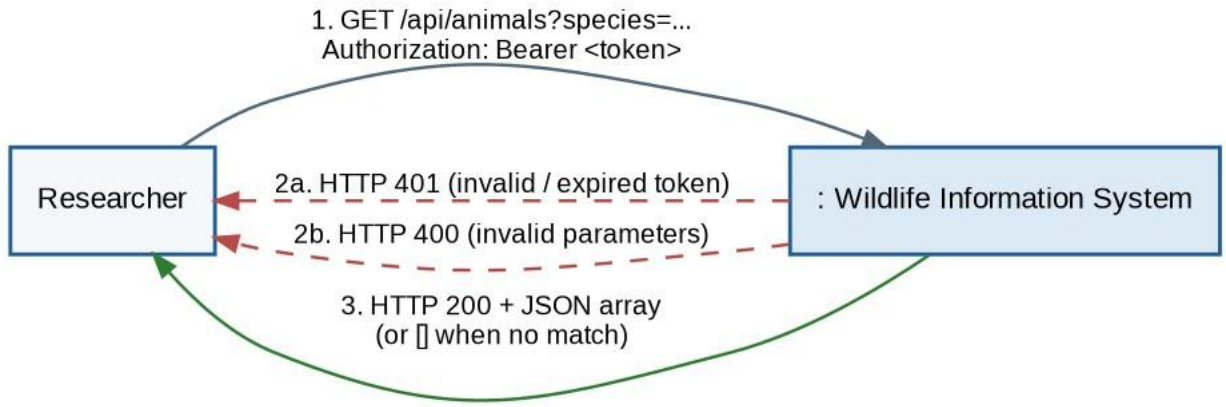


*Figure 3.4: System sequence for querying animal data through the API.*

## Main success and exception paths

1. The researcher sends a GET reques  t with query parameters and a Bearer JWT token.
2. WIS validates the token. An invalid or expired token results in HTTP 401 Unauthorized.
3. WIS validates the query parameters. Invalid parameters result in HTTP 400 Bad Request.
4. WIS searches the animal and species data. Matching records are returned as a JSON array; no matches pro ducean empty array.
5. A successful query returns HTTP 200 with the JSON response.

## 3.4 Activity Diagram

The activity diagram expands the API query flow to show validation decisions and the two principal error outcomes.

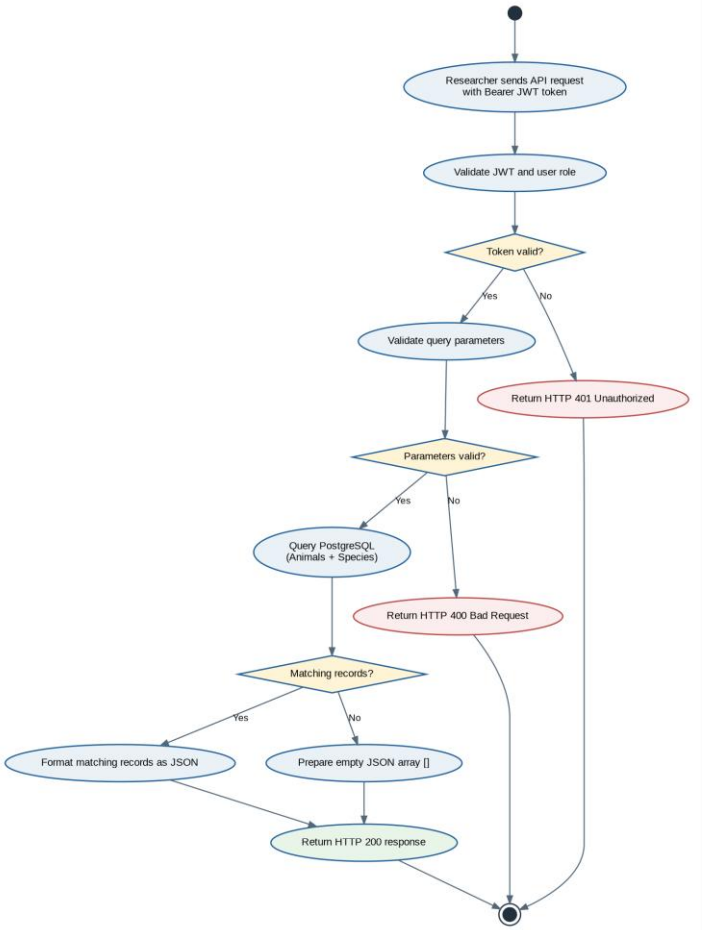



*Figure 3.5: Activity diagram — authenticated API query and response flow.*